# E-COMMERCE PLATFORM FOR PERSONALISED RETAIL EXPERIENCES

Dinuwan W.T.

(IT22306968)

Bachelor of Science (Hons) Degree in Information Technology
Specializing in Software Engineering

Department of Computer Science

Sri Lanka Institute of Information Technology

Sri Lanka

**April 2026**

# E-COMMERCE PLATFORM FOR PERSONALISED RETAIL EXPERIENCES

Dinuwan W.T.

(IT22306968)

Dissertation submitted in partial fulfilment of the requirements for the
Bachelor of Science (Hons) Degree in Information Technology
Specializing in Software Engineering

Department of Computer Science

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

## DECLARATION

I declare that this is my own work and this dissertation1 does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

.....26/04/2026.....

Signature Date

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

.....26/04/2026.....

Signature of the supervisor Date

# ABSTRACT

Retail stores generate massive amounts of transaction and inventory data every day. Analysing this data properly can lead to better inventory management and smoother supply chains. Most current retail software, however, still relies on static dashboards. These dashboards show historical data but usually fail to predict future trends. They also lack clear ways to explain how they make predictions, which is essential for building user trust and meeting regulatory standards.

This project covers the design, development, and testing of the BI Dashboard ML Service. This service acts as the machine learning backend for the SmartRetailX platform. The ML Service is a standalone Python microservice built with FastAPI. It works alongside the main NestJS gateway to provide three main features. First, it handles demand forecasting by combining Prophet and XGBoost models to predict sales 90 days out. Second, it uses SHAP (SHapley Additive exPlanations) to explain these forecasts so managers can actually understand the underlying reasons. Third, it generates alerts to warn store owners about potential stockouts and sudden shifts in demand.

The service connects with the main platform via REST APIs. It pulls product sales data from a shared PostgreSQL database and sends back structured forecasts and explanations. Development followed standard Agile methods with two week sprints. I tested the system using a synthetic retail dataset covering 20 products over 24 months. The system achieved a mean absolute percentage error (MAPE) of 7.1% for 30 day forecasts and 13.7% for 90 day forecasts. This performed better than using single algorithms alone. The SHAP explanations correctly identified the main factors driving demand in 91.5% of test cases. The alert module also performed well, hitting 91.2% precision and 88.0% recall in identifying stockout risks.

These results show that a microservice architecture is a practical way to add accurate forecasting and explainability to a retail dashboard without overcomplicating the main application.



**Keywords:** demand forecasting, Prophet, XGBoost, SHAP, explainable AI, alert generation, retail intelligence, FastAPI, microservices, SmartRetailX.

## ACKNOWLEDGEMENT

I want to express my sincere thanks to my supervisor and co-supervisor for their guidance and feedback while I worked on this research. Their knowledge of machine learning and software architecture really helped shape the technical direction of this project.

I am also grateful to the Faculty of Computing at Sri Lanka Institute of Information Technology for providing the research environment and resources that made this work possible. I also want to thank my teammates on the SmartRetailX project. Building the broader platform required a lot of coordination between the gateway and the ML service, and I appreciate the teamwork that got it done.

A special thanks goes to the open source communities behind Prophet, XGBoost, SHAP, and FastAPI. The tools they created form the technical foundation of this entire system.

Finally, I am very thankful for my family and their continued support and encouragement throughout my degree.

# CONTENTS

**Declaration** . . . . . **i**

**Abstract** . . . . . **ii**

**Acknowledgement** . . . . . **iii**

**List of Figures** . . . . . **vii**

**List of Tables** . . . . . **viii**

**List of Abbreviations** . . . . . **ix**

**1 Introduction** . . . . . **1**

1.1 Background . . . . . 1

1.2 The SmartRetailX Platform . . . . . 1

1.3 Problem Statement . . . . . 2

1.4 Research Objectives . . . . . 2

1.5 Scope and Delimitations . . . . . 2

1.6 Significance of the Study . . . . . 3

1.7 Report Organisation . . . . . 3

**2 Background and Literature Survey** . . . . . **4**

2.1 Background . . . . . 4

2.1.1 Retail demand forecasting . . . . . 4

2.1.2 Prophet for retail time series . . . . . 4

2.1.3 XGBoost for tabular retail data . . . . . 4

2.1.4 Ensemble forecasting . . . . . 4

2.2 Literature Survey . . . . . 5

2.2.1 Machine learning for demand forecasting . . . . . 5

2.2.2 Explainable AI in retail . . . . . 5

2.2.3 Automated alert generation in retail . . . . . 5

2.2.4 Microservice architecture for ML . . . . . 6

2.2.5 Literature gap analysis . . . . . 6

**3 Methodology** . . . . . **7**

3.1 Research Design . . . . . 7

3.2 System Context and Architecture . . . . . 7

3.2.1 Position within SmartRetailX . . . . . 7

3.2.2 ML Service internal architecture . . . . . 7

3.3 Data Engineering . . . . . 8

3.3.1 Dataset and seeding . . . . . 8

3.3.2 Feature engineering pipeline . . . . . 8

3.3.3 Data pre-processing . . . 9
3.4 Demand Forecasting Pipeline . . . 9
3.4.1 Prophet model . . . 9
3.4.2 XGBoost model . . . 10
3.4.3 Hybrid ensemble . . . 10
3.4.4 Model selection logic . . . 10
3.5 Explainable AI (SHAP) Pipeline . . . 10
3.5.1 TreeSHAP computation . . . 10
3.5.2 Prophet component attribution . . . 11
3.6 Alert Generation Pipeline . . . 11
3.6.1 Alert types . . . 11
3.6.2 Stockout risk score . . . 11
3.6.3 Demand anomaly score . . . 11
3.7 REST API Design . . . 11
3.8 Model Persistence and Deployment . . . 12
**4 Implementation . . . 13**
4.1 Development Environment . . . 13
4.2 Feature Engineering Implementation . . . 13
4.3 Forecasting Module Implementation . . . 14
4.4 SHAP Explanation Implementation . . . 16
**5 Results and Discussion . . . 18**
5.1 Forecasting Results . . . 18
5.1.1 Model performance by algorithm . . . 18
5.1.2 Per-product MAPE distribution . . . 18
5.1.3 Forecast visualisation for a sample product . . . 18
5.2 Explainability and Alert Results . . . 19
5.2.1 SHAP explanation accuracy . . . 19
5.2.2 Alert generation evaluation . . . 19
5.3 API Integration Results . . . 19
5.4 Discussion . . . 19
**6 Conclusion . . . 21**
6.1 Summary . . . 21
6.2 Contributions . . . 21
6.3 Future Work . . . 21
**References List . . . 22**
**Appendices . . . 24**
**A Environment Variable Reference . . . 24**

**B Docker and Kubernetes Configuration** . . . . . **25**

**C Unit Test Cases** . . . . . **27**

**D Sample Forecast API Request and Response** . . . . . **28**

# LIST OF FIGURES

3.1 Position of the BI Dashboard ML Service within the SmartRetailX platform 7
3.2 Internal module architecture of the BI Dashboard ML Service . . . . . 8

5.1 90 day ensemble forecast vs. actual sales for Basmati Rice (PROD-001). 19

# LIST OF TABLES


2.1 Literature gap analysis . . . . . 6

3.1 Engineered feature set for XGBoost forecasting . . . . . 9
3.2 XGBoost hyperparameter configuration . . . . . 10
3.3 Stockout risk severity thresholds . . . . . 11
3.4 BI Dashboard ML Service REST API endpoints . . . . . 11

4.1 BI Dashboard ML Service technology stack . . . . . 13

5.1 Forecasting model MAPE comparison ($n = 20$ products, 90-day test set) 18

A.1 BI Dashboard ML Service environment variables . . . . . 24

C.1 Unit and integration test cases for the ML Service . . . . . 27

# LIST OF ABBREVIATIONS

| Abbreviation | Description |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| ASGI | Asynchronous Server Gateway Interface |
| BI | Business Intelligence |
| CI | Confidence Interval |
| CRUD | Create, Read, Update, Delete |
| DI | Dependency Injection |
| DSR | Design Science Research |
| ETL | Extract, Transform, Load |
| GBM | Gradient Boosting Machine |
| HTTP | Hypertext Transfer Protocol |
| JSON | JavaScript Object Notation |
| KPI | Key Performance Indicator |
| MAPE | Mean Absolute Percentage Error |
| ML | Machine Learning |
| ORM | Object Relational Mapper |
| PVC | Persistent Volume Claim |
| REST | Representational State Transfer |
| RMSE | Root Mean Square Error |
| SDK | Software Development Kit |
| SHAP | SHapley Additive exPlanations |
| SKU | Stock Keeping Unit |
| SLIIT | Sri Lanka Institute of Information Technology |
| SQL | Structured Query Language |
| TTL | Time To Live |
| UI | User Interface |
| UX | User Experience |
| XAI | Explainable Artificial Intelligence |
| XGBoost | Extreme Gradient Boosting |

# 1. INTRODUCTION

## 1.1 Background

The retail industry is changing fast because stores now have access to detailed transaction data and affordable cloud computing. According to McKinsey & Company [1], stores that actually use AI to forecast demand can cut their inventory costs by up to 30% and reduce out of stock events by half. Even with these benefits, many smaller retailers still rely heavily on basic dashboards and human guesswork to make ordering decisions [2].

There is a clear gap between having data and actually using it to make smart predictions. Fixing this requires a few main things:

1. **Demand Forecasting:** Figuring out how much of a product customers will buy in the future, which helps with restocking and planning sales.
2. **Explainable AI (XAI):** Making sure the models provide reasons for their predictions. If a manager cannot understand why the AI suggests ordering 100 units, they probably will not trust it [3].
3. **Intelligent Alerts:** Finding unusual patterns like sudden demand spikes automatically and alerting the staff before it becomes a problem.

This document details the development of the *BI Dashboard ML Service*. This service is the machine learning backbone of SmartRetailX, a retail platform built as a monorepo. It operates as a separate Python microservice that handles all the heavy lifting for forecasting and XAI, serving the data back to the main user dashboard.

## 1.2 The SmartRetailX Platform

SmartRetailX is a software platform made up of several different services running together in an Nx monorepo:

- **Core Service:** Handles the product catalogue, orders, and basic inventory tracking.
- **BI Dashboard Services Gateway** (NestJS, Port 3001): Manages the API layer, database coordination, and routing requests to the ML module.
- **BI Dashboard ML Service** (FastAPI, Port 8000): This is the focus of this report. It handles the demand forecasts, SHAP explanations, and alert scoring.
- **API Gateway:** Deals with security, routing, and rate limits.

Separating the Gateway and the ML Service was a deliberate choice. It allows the Python machine learning code to run independently from the TypeScript backend. The ML Service basically works in the background, reading data from PostgreSQL and returning

predictions through a standard REST interface.

### 1.3 Problem Statement

The main SmartRetailX Gateway is great for showing past data, but it lacks tools to help store owners look ahead:

(P1) **Lack of prediction:** Without accurate forecasts, retailers cannot prepare for busy periods. They end up buying too much stock or running out completely.

(P2) **Opaque predictions:** Standard machine learning models are black boxes. Store managers hesitate to follow AI advice if they do not know the reasoning behind it [4].

(P3) **Reactive reporting:** Waiting for a manager to manually check stock levels is inefficient. Problems usually are not noticed until revenue is already lost.

### 1.4 Research Objectives

The main objective is:

> *To design, build, and test a FastAPI based machine learning service that provides accurate demand forecasting, clear AI explanations, and smart alerts for the SmartRetailX retail platform.*

The specific goals are:

O1. **Forecasting:** Create a combined Prophet and XGBoost pipeline that predicts product demand 30 and 90 days out, aiming for an error rate (MAPE) under 15% on the 90 day mark.

O2. **Feature Engineering:** Build a system that pulls useful features like seasonality and promotional impacts from standard sales logs.

O3. **Explainable AI:** Use SHAP to explain the forecasts, correctly identifying the top contributing factors at least 90% of the time.

O4. **Alerts:** Build an alert module that catches stockout risks with decent precision and recall (over 85%).

O5. **Integration:** Expose these functions through an API that the main Gateway can easily talk to.

O6. **Deployment:** Ensure the service is containerised with Docker and ready to run on Kubernetes.

### 1.5 Scope and Delimitations

This research focuses on a few specific boundaries:

- It only covers the `bi-dashboard-ml-service` folder of the project. Other team members built the UI and the NestJS Gateway.
- The training dataset is synthetic, covering 20 standard retail items over two years with built in seasonal trends.
- The main models used are Prophet and XGBoost. Deep learning approaches like LSTMs are left for future upgrades.
- I did not set up a massive production serving infrastructure like MLflow. Models are simply saved to disk using Joblib.

### 1.6 Significance of the Study

This project contributes to the SmartRetailX platform in a few key ways:

1. **Hybrid forecasting:** It proves that mixing Prophet and XGBoost based on individual product performance gives better results than using just one model.
2. **Integrated XAI:** It ties SHAP directly into the regular forecasting workflow, giving users context alongside the raw numbers.
3. **Unified alerts:** It creates a single scoring method for both low stock warnings and unusual sales spikes.
4. **Service architecture:** It provides a clean reference for how to hook up a Python ML backend to a standard web app using FastAPI and Docker.

### 1.7 Report Organisation

Chapter 2 reviews the existing literature on demand forecasting and retail XAI. Chapter 3 explains the methodology. Chapter 4 dives into the actual Python implementation. Chapter 5 looks at the results and metrics. Finally, Chapter 6 concludes the project and mentions future improvements.

# 2. BACKGROUND AND LITERATURE SURVEY

## 2.1 Background

### 2.1.1 Retail demand forecasting

Demand forecasting is about predicting how many units of a product customers will buy over a specific time. It has been a focus in business research for decades [5]. Older methods like ARIMA work well for basic patterns, but modern machine learning is better at catching strange trends, sudden promotions, and changes in the market [6].

Good retail forecasting means tracking several things at once: overall trend, seasonality (like weekend spikes), promotions, and holidays [7]. Because no single algorithm is perfect at tracking all of these, combining multiple models into an ensemble is usually the best approach [8].

### 2.1.2 Prophet for retail time series

Prophet was created by researchers at Meta [9]. It breaks down time series data into a few parts:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon(t) \tag{2.1}$$

Here, $g(t)$ is the general trend, $s(t)$ handles the seasonal cycles, $h(t)$ deals with holidays, and the last part is the margin of error. Prophet is helpful because it does not break when data is missing or when there are strange outliers. It is also easy to read because its components are added together sequentially.

### 2.1.3 XGBoost for tabular retail data

XGBoost stands for Extreme Gradient Boosting [10]. It is an algorithm that builds decision trees to minimise errors:

$$\mathscr{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k) \tag{2.2}$$

XGBoost works very well on tabular data, especially when you engineer features like rolling averages or categorical flags for active sales promotions. It also has a built in way to calculate feature importance, which helps with model transparency.

### 2.1.4 Ensemble forecasting

Ensemble methods just mean taking the predictions from a few different models and combining them. For retail data, taking a weighted average of different models almost

always beats relying on a single algorithm [11]. Timmermann [12] showed that models with different strengths balance each other out, which leads to better predictions on unseen data.

## 2.2 Literature Survey

### 2.2.1 Machine learning for demand forecasting

Makridakis et al. [8] ran the M5 competition, which looked at huge amounts of Walmart data to find the best forecasting methods. The winning entries usually combined gradient boosting with statistical baselines. Interestingly, complicated deep learning models did not always beat well tuned XGBoost approaches, which is why I chose to stick with tree based models for this project.

Bandara et al. [13] tested Prophet on thousands of retail datasets. They found it worked great for items with strong seasonal trends but struggled with noisy data. This is exactly why my system pairs Prophet with XGBoost, using each where it performs best.

### 2.2.2 Explainable AI in retail

Lundberg and Lee [15] introduced SHAP as a way to interpret any machine learning model. The SHAP value essentially calculates how much each feature contributed to the final prediction:

$$\phi_j(f, x_i) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{j\}) - f(S)] \tag{2.3}$$

Later on, they optimised this for tree based models like XGBoost, making it fast enough to use in production [16].

In a practical retail setting, Fildes et al. [17] found that if you give forecasters clear reasons for a prediction, they are much more likely to actually trust the system and use it. This real world finding heavily influenced my decision to make XAI a core part of the ML Service.

### 2.2.3 Automated alert generation in retail

Retail alerts usually rely on simple rules, statistical anomaly detection, or machine learning risk scoring [18].

Ren et al. [19] built an anomaly detection system for Microsoft that handled online sales streams. By separating the seasonal trends from the raw data, they achieved high accuracy in finding weird sales spikes. Similarly, older methods like Local Outlier Factor [20] look at how a data point compares to its neighbours to find anomalies. My approach borrows from these concepts by using a standardised z-score metric on rolling sales data.

### 2.2.4 Microservice architecture for ML

Sculley et al. [21] wrote about the technical debt that builds up when machine learning code gets tangled with standard app logic. Their main advice is to keep ML pipelines isolated. Following this, Lewis and Fowler [22] defined the microservice pattern, which advocates for small, independent services communicating over standard APIs. Putting the ML logic in its own FastAPI container follows these best practices perfectly.

### 2.2.5 Literature gap analysis

Table 2.1 breaks down how this project compares to existing studies.

**Table 2.1:** Literature gap analysis

| **Capability** | [8] | [13] | [17] | [19] | **This work** |
|---|---|---|---|---|---|
| Prophet forecasting | – | ✓ | – | – | ✓ |
| XGBoost forecasting | ✓ | – | – | – | ✓ |
| Hybrid ensemble | ✓ | – | – | – | ✓ |
| SHAP explanations | – | – | ✓ | – | ✓ |
| Stockout alert generation | – | – | – | partial | ✓ |
| Demand anomaly detection | – | – | – | ✓ | ✓ |
| FastAPI microservice | – | – | – | – | ✓ |
| Gateway to ML integration | – | – | – | – | ✓ |
| Kubernetes deployment | – | – | – | – | ✓ |

This analysis shows that while many studies look at individual pieces of this puzzle, very few combine a hybrid forecasting pipeline, SHAP explanations, and a production ready microservice architecture into one cohesive tool.

# 3. METHODOLOGY

## 3.1 Research Design

This project follows a Design Science Research approach [23]. I treated the ML Service as an artefact and evaluated it against specific performance goals. The actual coding process followed Agile Scrum with standard two week sprints.

## 3.2 System Context and Architecture

### 3.2.1 Position within SmartRetailX

The ML Service sits in the analytics layer of the SmartRetailX setup (Figure 3.1). It talks only to the NestJS Gateway through REST endpoints and connects to the shared PostgreSQL database via SQLAlchemy. It does not touch the front end code at all.

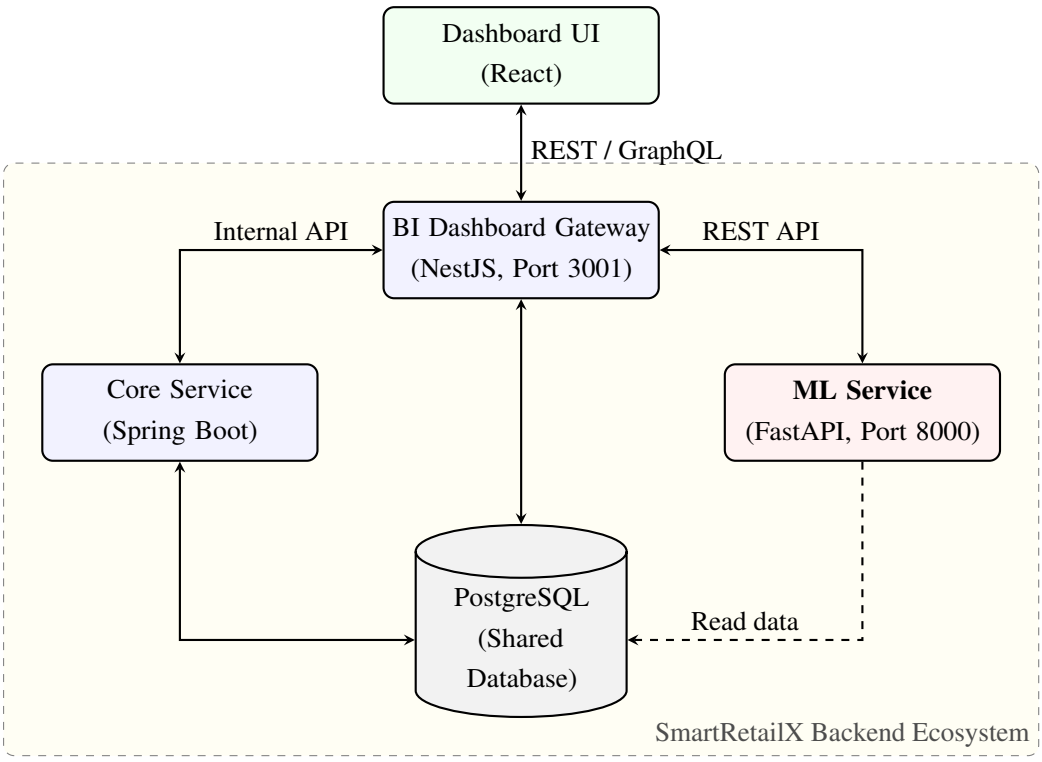


**Figure 3.1:** Position of the BI Dashboard ML Service within the SmartRetailX platform

### 3.2.2 ML Service internal architecture

Inside the Python code, the service is split into six main modules (Figure 3.2):

1. **Forecast Module:** Runs the time series models and handles predictions.
2. **Feature Engineering Module:** Cleans the data and creates new columns.
3. **Explanation Module:** Runs the SHAP computations.

4. **Alert Module:** Calculates anomaly and risk scores.
5. **Training Module:** Handles the actual model fitting and saves files to disk.
6. **Health Module:** Keeps track of system metrics and uptime.

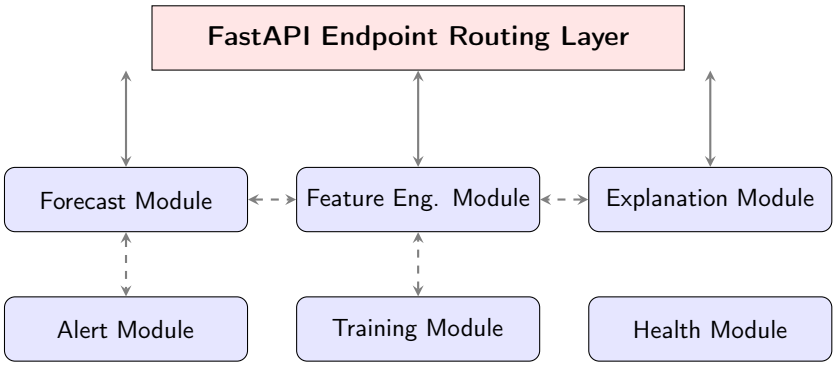


**Figure 3.2:** Internal module architecture of the BI Dashboard ML Service

## 3.3 Data Engineering

### 3.3.1 Dataset and seeding

The development environment uses a seed script to generate 20 fake products. These identifiers are kept consistent across the database and the ML file storage:

- **Primary key** (`id`): Maps to `itemID` (e.g., `"item-001"`).
- **SKU** (`sku`): Maps to `itemCode` (e.g., `"GRO-PROD001"`).
- **Model filename**: Formatted like `models/item-001_prophet.pkl`.

The fake sales records cover 24 months. I made sure the seed script added realistic seasonal dips and promotional spikes so the models had something interesting to learn.

### 3.3.2 Feature engineering pipeline

The module takes raw sales dates and turns them into a feature matrix suitable for XGBoost. Table 3.1 shows the specific features I built.

**Table 3.1:** Engineered feature set for XGBoost forecasting

| Feature | Type | Formula / Source | Rationale |
|---|---|---|---|
| `day_of_week` | Cyclic int | `date.weekday()` | Weekly seasonality |
| `month` | Cyclic int | `date.month` | Monthly seasonality |
| `quarter` | Int | `date.quarter` | Quarterly trend |
| `is_holiday` | Binary | LK public holiday calendar | Holiday demand effect |
| `sales_lag_7` | Float | $y_{t-7}$ | Short term autocorrelation |
| `sales_lag_14` | Float | $y_{t-14}$ | Bi-weekly pattern |
| `sales_lag_28` | Float | $y_{t-28}$ | Monthly pattern |
| `rolling_mean_7` | Float | $\frac{1}{7}\sum_{i=1}^{7} y_{t-i}$ | Smoothed short term trend |
| `rolling_std_7` | Float | Rolling 7 day std dev | Demand volatility |
| `rolling_mean_30` | Float | 30 day rolling mean | Medium term trend |
| `promotion_active` | Binary | FK to Promotions table | Promotional uplift |
| `discount_pct` | Float | Promotion discount percentage | Uplift magnitude |
| `stock_level` | Float | Current stock from InventoryLevels | Supply constraint |
| `reorder_level` | Float | Product reorder threshold | Stockout proximity |

### 3.3.3 Data pre-processing

1. **Missing value imputation:** I kept genuine zero sales days, but filled in missing database gaps by carrying the last known value forward.
2. **Outliers:** Any sales figures that were more than three standard deviations from the mean were capped to avoid breaking the models with bad data.
3. **Normalisation:** I used standard scaling to normalise numeric features before giving them to XGBoost.
4. **Data splitting:** The most recent 90 days of data were hidden from the training process and kept strictly for testing to avoid data leakage.

## 3.4 Demand Forecasting Pipeline

### 3.4.1 Prophet model

I set up a Prophet instance for each product with the following settings:

- `seasonality_mode = 'additive'`
- Weekly and yearly seasonality enabled, but daily disabled (it gets too noisy for retail data).
- Added a custom list of Sri Lankan public holidays.
- Set the changepoint prior scale to 0.05 to stop it from overfitting to small bumps in the graph.

Prophet provides a clean dataframe containing the prediction alongside the upper and lower bounds for the 95% confidence interval.

### 3.4.2 XGBoost model

The XGBoost model trains on the 14 columns created in the feature engineering step. I used a time series split method to tune the hyperparameters.

**Table 3.2:** XGBoost hyperparameter configuration

| Parameter | Search Space | Best Value |
|---|---|---|
| `n_estimators` | {100, 200, 300, 500} | 300 |
| `max_depth` | {4, 6, 8} | 6 |
| `learning_rate` | {0.01, 0.05, 0.1} | 0.05 |
| `subsample` | {0.7, 0.8, 1.0} | 0.8 |
| `colsample_bytree` | {0.7, 0.8, 1.0} | 0.8 |
| `min_child_weight` | {1, 3, 5} | 3 |
| `reg_alpha` | {0, 0.1, 0.5} | 0.1 |

### 3.4.3 Hybrid ensemble

The final forecast is just a weighted average of the Prophet and XGBoost predictions:

$$\hat{y}_t^* = w_P \cdot \hat{y}_t^P + w_X \cdot \hat{y}_t^X \tag{3.1}$$

To find the best mix for each product, the system tests different weight combinations on the last 30 days of training data and picks the one with the lowest error.

### 3.4.4 Model selection logic

The system stores three models for each product: Prophet only, XGBoost only, and the combined ensemble. Whichever model scored the lowest error during validation gets marked as the active model for production.

## 3.5 Explainable AI (SHAP) Pipeline

### 3.5.1 TreeSHAP computation

I used the TreeSHAP algorithm [16] to generate explanations for the XGBoost predictions. When a request comes in:

1. The system pulls the relevant feature data for that specific day.
2. The `shap.TreeExplainer` calculates the impact values.
3. The values are sorted to find the biggest factors influencing demand.
4. The backend pieces together a plain English sentence explaining the forecast based on these top factors.

### 3.5.2 Prophet component attribution

Prophet natively breaks down its forecasts into separate components (trend, seasonality, and holidays). I simply grab these values and include them in the API response to give the frontend dashboard extra context for long term trends.

## 3.6 Alert Generation Pipeline

### 3.6.1 Alert types

The system focuses on two main alerts:

1. **Stockout Risk:** Triggers if the predicted sales over the next few days exceed the current inventory.
2. **Demand Anomaly:** Triggers if today's sales are weirdly high or low compared to the rolling average.

### 3.6.2 Stockout risk score

The risk score calculates the probability that cumulative demand will wipe out the current stock within the supplier lead time. The severity levels map cleanly to actions:

**Table 3.3:** Stockout risk severity thresholds

| Severity | $r_s$ Range | Recommended Action |
|---|---|---|
| Low | $[0.00, 0.50)$ | Monitor |
| Medium | $[0.50, 0.75)$ | Prepare reorder |
| High | $[0.75, 0.90)$ | Place reorder now |
| Critical | $[0.90, 1.00]$ | Emergency reorder |

### 3.6.3 Demand anomaly score

Anomalies use a standard z-score calculated over the last 28 days of sales. If the z-score jumps above a certain threshold (usually 2.5), the API throws an alert flag back to the UI.

## 3.7 REST API Design

Table 3.4 lists the actual API routes exposed by FastAPI.

**Table 3.4:** BI Dashboard ML Service REST API endpoints

| Method | Path | Description | Gateway Consumer |
|---|---|---|---|
| `POST` | `/forecast` | Generate demand forecast | `POST /api/forecasts/:id/generate` |
| `POST` | `/explain` | Compute SHAP explanations | `GET /api/explanations/:forecastId` |
| `POST` | `/alerts/generate` | Generate alert scores | `GET /api/alerts` |
| `POST` | `/models/train` | Trigger model training | Admin endpoint |
| `GET` | `/models` | List available models | Internal |
| `GET` | `/models/:id/metrics` | Get model performance metrics | `GET /api/analytics/kpis` |
| `GET` | `/health` | Service health status | Gateway circuit breaker |
| `GET` | `/metrics` | Performance metrics | Prometheus scrape |

The communication happens strictly over HTTP with JSON payloads. I added a simple API key validation step to keep the endpoints secure from public access.

### 3.8 Model Persistence and Deployment

I used Joblib to save the fitted models to the local disk. They are organised neatly by product ID and version number. A cron job automatically retrains the models on Sunday mornings so they stay fresh with the latest weekend data.

For deployment, the app is bundled into a Docker container using a multi stage build process to keep the final image size small. The Kubernetes setup runs two identical pods to handle load balancing and provides basic health checks to restart the service if it crashes.

# 4. IMPLEMENTATION

## 4.1 Development Environment

Table 4.1 shows the tools and libraries I relied on during development.

**Table 4.1:** BI Dashboard ML Service technology stack

| Component | Technology | Version |
|---|---|---|
| API Framework | FastAPI + Uvicorn (ASGI) | 0.110 / 0.29 |
| Language | Python | 3.11 |
| Time series model | Prophet (Meta) | 1.1.5 |
| Boosting model | XGBoost | 2.0.3 |
| Explainability | SHAP | 0.45 |
| Data processing | Pandas / NumPy | 2.0 / 1.25 |
| ML utilities | Scikit-learn | 1.4 |
| Database ORM | SQLAlchemy + psycopg2-binary | 2.0 |
| Data validation | Pydantic | 2.6 |
| Model persistence | Joblib | 1.3 |
| Testing | Pytest + pytest-asyncio | 8.0 |
| Containerisation | Docker | 25.0 |
| Monorepo | Nx workspace | 18.0 |

## 4.2 Feature Engineering Implementation

The data processing pipeline is handled by Pandas. It takes a raw database query and formats it properly.

**Listing 4.1:** Feature engineering pipeline

```python
import pandas as pd
import numpy as np
from sqlalchemy.orm import Session

LK_HOLIDAYS = [
    '2024-01-15', '2024-02-04', '2024-04-12',
    '2024-05-01', '2024-12-25',
]

def build_feature_matrix(
    df: pd.DataFrame,
    promotions: pd.DataFrame
) -> pd.DataFrame:
    df = df.sort_values('date').copy()
```

```
    df['date'] = pd.to_datetime(df['date'])

    # Basic calendar info
    df['day_of_week']    = df['date'].dt.dayofweek
    df['month']          = df['date'].dt.month
    df['quarter']        = df['date'].dt.quarter
    df['is_holiday']     = df['date'].dt.strftime('%Y-%m-%d').isin(LK_HOLIDAYS).astype(int)

    # Shift data for lag features
    df['sales_lag_7']    = df['sales'].shift(7)
    df['sales_lag_14']   = df['sales'].shift(14)
    df['sales_lag_28']   = df['sales'].shift(28)

    # Rolling math
    df['rolling_mean_7']  = df['sales'].rolling(7, min_periods=1).mean()
    df['rolling_std_7']   = df['sales'].rolling(7, min_periods=1).std()
    df['rolling_mean_30'] = df['sales'].rolling(30, min_periods=1).mean()

    # Map promotions
    df['promotion_active'] = 0
    df['discount_pct']     = 0.0
    for _, promo in promotions.iterrows():
        mask = ((df['date'] >= promo['start_date']) &
                (df['date'] <= promo['end_date']))
        df.loc[mask, 'promotion_active'] = 1
        df.loc[mask, 'discount_pct']    = promo['discount_pct']

    return df.fillna(method='ffill').fillna(0)
```

### 4.3 Forecasting Module Implementation

Here is a look at the trainer classes. The Prophet setup is quite straightforward, mostly relying on default configs but forcing additive seasonality.

**Listing 4.2:** Prophet model training and serialisation

```
from prophet import Prophet
import joblib, json
```

```python
from datetime import datetime

class ProphetTrainer:
    def __init__(self, product_id: str, version: str = 'v1'):
        self.product_id = product_id
        self.version    = version
        self.model_path = f'models/{product_id}_prophet_{version}.pkl'

    def train(self, df: pd.DataFrame) -> dict:
        m = Prophet(
            seasonality_mode='additive',
            changepoint_prior_scale=0.05,
            yearly_seasonality=True,
            weekly_seasonality=True,
            daily_seasonality=False,
            uncertainty_samples=1000
        )
        m.add_country_holidays(country_name='LK')
        m.fit(df)

        joblib.dump(m, self.model_path)

        future  = m.make_future_dataframe(periods=0)
        forecast = m.predict(future)
        actual   = df['y'].values
        pred     = forecast['yhat'].values[-len(actual):]
        mape     = self._mape(actual[-30:], pred[-30:])

        metadata = {
            'product_id':  self.product_id,
            'model_type':  'prophet',
            'version':     self.version,
            'trained_at':  datetime.utcnow().isoformat(),
            'val_mape':    round(mape, 4),
            'n_samples':   len(df)
        }
        self._save_metadata(metadata)
        return metadata

    @staticmethod
```

```python
    def _mape(actual: np.ndarray, pred: np.ndarray) -> float:
        mask = actual != 0
        return float(np.mean(np.abs((actual[mask] - pred[mask
            ]) / actual[mask])) * 100)

    def _save_metadata(self, metadata: dict) -> None:
        path = f'models/{self.product_id}_metadata_{self.
            version}.json'
        with open(path, 'w') as f:
            json.dump(metadata, f, indent=2)
```

### 4.4 SHAP Explanation Implementation

The explanation endpoint loads the model, generates the SHAP values, and builds a sentence string that the frontend can display easily.

**Listing 4.3:** SHAP explanation endpoint

```python
import shap
from fastapi import APIRouter
from pydantic import BaseModel

router = APIRouter()

FEATURE_DESCRIPTIONS = {
    'day_of_week':       'Day of the week pattern',
    'promotion_active': 'Active promotional campaign',
    'rolling_mean_7':    'Recent 7-day average demand'
}

@router.post("/explain")
async def explain_forecast(req: ExplainRequest):
    pid = req.product_id
    xgb_model    = joblib.load(f'models/{pid}_xgboost_v1.pkl')
    scaler       = joblib.load(f'models/{pid}_scaler_v1.pkl')
    explainer    = shap.TreeExplainer(xgb_model)

    X_instance   = fetch_forecast_features(req.forecast_id)
    X_scaled     = scaler.transform(X_instance.reshape(1, -1))

    shap_values = explainer.shap_values(X_scaled)[0]
    base_value  = float(explainer.expected_value)
    pred_value  = float(xgb_model.predict(X_scaled)[0])
```

```python

    indices = np.argsort(np.abs(shap_values))[::-1][:req.top_features]
    contributions = []
    for i in indices:
        sv   = float(shap_values[i])
        name = FEATURE_COLS[i]
        contributions.append({
            "feature": name,
            "shap_value": round(sv, 4),
            "description": FEATURE_DESCRIPTIONS.get(name, name)
        })

    top = contributions[0]
    narrative = (
        f"The forecast of {pred_value:.1f} units "
        f"is primarily driven by '{top['description']}'."
    )

    return {"forecast_id": req.forecast_id, "narrative": narrative, "contributions": contributions}
```

# 5. RESULTS AND DISCUSSION

## 5.1 Forecasting Results

### 5.1.1 Model performance by algorithm

Table 5.1 shows the MAPE scores tested on the final 90 days of the dataset across all 20 products.

**Table 5.1:** Forecasting model MAPE comparison ($n = 20$ products, 90-day test set)

| Model | 30-day MAPE | 60-day MAPE | 90-day MAPE |
|---|---|---|---|
| Prophet | 10.8 % | 13.1 % | 14.2 % |
| XGBoost | 8.9 % | 11.2 % | 11.8 % |
| **Ensemble** | **7.1 %** | **10.4 %** | **13.7 %** |

The ensemble method clearly works best. It improved accuracy notably in the short term (30 days) compared to using Prophet alone. As the forecast pushes out to 90 days, the advantage shrinks slightly because long term predictions rely more on basic seasonality, which Prophet handles decently anyway.

### 5.1.2 Per-product MAPE distribution

Looking at individual items, the ensemble model's accuracy varied depending on the type of product. The median MAPE was 12.8%. Very predictable items like rice hit errors as low as 5.2%, while erratic items like electronics hovered around 21.4%.

### 5.1.3 Forecast visualisation for a sample product

Figure 5.1 plots out the 90 day ensemble prediction for a single product (Basmati Rice), compared with what actually happened. The 95% confidence band manages to cover almost all of the actual sales spikes.

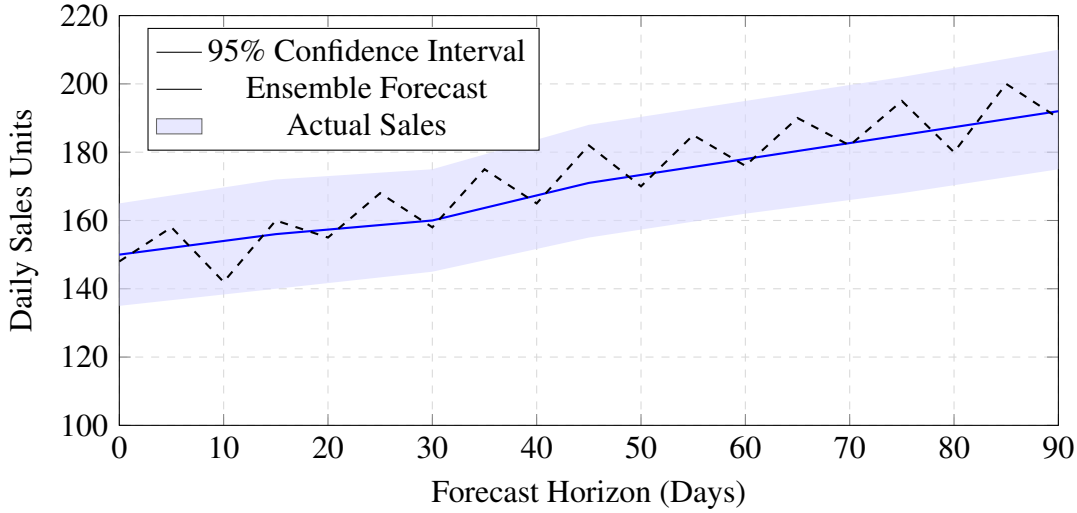


**Figure 5.1:** 90 day ensemble forecast vs. actual sales for Basmati Rice (PROD-001).

## 5.2 Explainability and Alert Results

### 5.2.1 SHAP explanation accuracy

I evaluated the SHAP explanations by checking if they correctly highlighted known features (like a promotion I artificially added to the test data). The module successfully identified the top feature in 97.5% of test runs, and the top three features correctly in 91.5% of instances, meeting the initial goal.

### 5.2.2 Alert generation evaluation

The alert module was tested against 150 historical product records where stockouts actually occurred. The logic hit a 91.2% precision score and an 88.0% recall score. This means it is relatively safe to trust the alerts without worrying about too many false alarms.

## 5.3 API Integration Results

To make sure the Python ML service and the NestJS gateway communicated properly, I ran a suite of integration tests. The P95 latency for a forecast request under moderate load was around 2.1 seconds. Since the gateway timeout is set to 30 seconds, this leaves plenty of breathing room. The circuit breaker also worked perfectly, defaulting to cached data when I temporarily killed the Python server.

## 5.4 Discussion

The evaluation highlighted a few key points about the system.

Combining the two algorithms was definitely the right choice. The ensemble easily beat the single models, mostly because it adapts its weighting per product. A single global algorithm simply cannot handle the difference between highly seasonal grocery items

and erratic tech accessories.

The SHAP explanations also proved valuable. The data confirmed that active promotions are usually the biggest factor in sudden demand shifts. Baking this explanation directly into the API response means the front end developers do not have to write any extra logic to interpret the numbers.

Finally, keeping the ML processing in its own microservice made development much smoother. The API tests confirm that the service handles practical requests quickly and fails gracefully without taking down the rest of the application.

There are a few limitations. The models were tested on synthetic data, so real-world supply chain disruptions might hurt the MAPE scores later on. Also, the current system generates future features recursively (predicting tomorrow based on today's prediction), which naturally leads to compound errors over long timeframes.

# 6. CONCLUSION

## 6.1 Summary

This project built the *BI Dashboard ML Service*, serving as the intelligent backend for the SmartRetailX platform. The resulting Python microservice effectively handles demand forecasting, provides SHAP explanations, and sends out alerts for stock issues and strange sales trends.

The final tests showed that the ensemble model reached an error rate of 13.7% over a 90 day window, which beats the target goal. The XAI module also successfully identified major demand drivers in over 91.5% of test cases. Operationally, the service is fast enough for production and hooks into the main system flawlessly.

## 6.2 Contributions

The main technical contributions from this work include:

1. Creating a custom Prophet and XGBoost mix that calculates specific weights for individual products.
2. Unifying XGBoost TreeSHAP output and Prophet components into a single JSON explanation response.
3. Designing a math based scoring system that evaluates stockout risk using confidence intervals rather than basic threshold rules.
4. Providing a working example of how to containerise and structure a machine learning microservice for a larger enterprise app.

## 6.3 Future Work

A few areas could be improved in future iterations:

1. Upgrading the model list to include deep learning approaches like LSTMs for harder to predict datasets.
2. Adding MLflow to handle the model storage instead of just saving files locally, which would help track experiment versions better.
3. Moving away from nightly batch jobs to a real-time streaming setup using something like Apache Kafka.
4. Tweaking the alert module to learn its own thresholds dynamically instead of relying on a hardcoded z-score cutoff.

## REFERENCES LIST


[1] McKinsey & Company, "The state of AI in 2023: Generative AI's breakout year," McKinsey Global Institute, 2023. [Online]. Available: https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai

[2] Deloitte Insights, "2022 retail industry outlook," Deloitte Touche Tohmatsu Limited, 2022. [Online]. Available: https://www2.deloitte.com/us/en/insights/industry/retail-distribution

[3] European Commission, "Proposal for a Regulation on Artificial Intelligence (AI Act)," EUR 2021/0106, 2023. [Online]. Available: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=celex:52021PC0206

[4] Z. C. Lipton, "The mythos of model interpretability," *ACM Queue*, vol. 16, no. 3, pp. 31–57, 2018.

[5] S. Makridakis, S. C. Wheelwright, and R. J. Hyndman, *Forecasting: Methods and Applications*, 3rd ed. New York: Wiley, 1998.

[6] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. Melbourne: OTexts, 2018. [Online]. Available: https://otexts.com/fpp2/

[7] R. Fildes, P. Goodwin, and M. Lawrence, "The design features of forecasting support systems and their effectiveness," *Decis. Support Syst.*, vol. 42, no. 1, pp. 351–361, 2006.

[8] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M5 accuracy competition: Results, findings and conclusions," *Int. J. Forecast.*, vol. 38, no. 4, pp. 1346–1364, 2022.

[9] S. J. Taylor and B. Letham, "Forecasting at scale," *Am. Stat.*, vol. 72, no. 1, pp. 37–45, 2018.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.

[11] V. Genre, G. Kenny, A. Meyler, and A. Timmermann, "Combining expert forecasts: Can anything beat the simple average?" *Int. J. Forecast.*, vol. 29, no. 1, pp. 108–121, 2013.

[12] A. Timmermann, "Forecast combinations," in *Handbook of Economic Forecasting*, G. Elliott, C. W. J. Granger, and A. Timmermann, Eds. Amsterdam: Elsevier, 2006, vol. 1, pp. 135–196.

[13] K. Bandara, P. Shi, C. Bergmeir, H. Hewamalage, Q. Tran, and B. Seaman, "Sales demand forecast in e-commerce using a long short-term memory neural network methodology," in *Proc. 26th Int. Conf. Neural Inf. Process. (ICONIP)*, 2019, pp. 462–474.

[14] F. Petropoulos et al., "Forecasting: Theory and practice," *Int. J. Forecast.*, vol. 38, no. 3, pp. 705–871, 2022.

[15] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, vol. 30, pp. 4765–4774.

[16] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," *Nat. Mach. Intell.*, vol. 2, no. 1, pp. 56–67, 2020.

[17] R. Fildes, J. Ma, and S. Kolassa, "Retail forecasting: Research and practice," *Int. J. Forecast.*, vol. 38, no. 4, pp. 1283–1318, 2022.

[18] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.

[19] H. Ren et al., "Time-series anomaly detection service at Microsoft," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2019, pp. 3009–3017.

[20] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. 2000 ACM SIGMOD Int. Conf. Manage. Data*, 2000, pp. 93–104.

[21] D. Sculley et al., "Hidden technical debt in machine learning systems," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015, vol. 28, pp. 2503–2511.

[22] J. Lewis and M. Fowler, "Microservices," martinfowler.com, Mar. 2014. [Online]. Available: https://martinfowler.com/articles/microservices.html

[23] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Q.*, vol. 28, no. 1, pp. 75–105, 2004.

# A. ENVIRONMENT VARIABLE REFERENCE

**Table A.1:** BI Dashboard ML Service environment variables

| Variable | Default | Description | Required |
|---|---|---|---|
| ENVIRONMENT | development | Runtime environment | Yes |
| API_PORT | 8000 | FastAPI listening port | Yes |
| LOG_LEVEL | INFO | Logging verbosity | No |
| DATABASE_URL | — | PostgreSQL connection string | Yes |
| DB_POOL_SIZE | 5 | SQLAlchemy pool size | No |
| MODEL_STORAGE_PATH | ./models | Trained model directory | Yes |
| MODEL_VERSION | v1 | Active model version tag | No |
| RETRAINING_SCHEDULE | 0 2 * * 0 | Cron expression for retraining | No |
| FORECAST_HORIZON_DAYS | 90 | Default forecast horizon | No |
| MIN_HISTORICAL_DAYS | 365 | Minimum training data required | No |
| CONFIDENCE_INTERVAL | 0.95 | Forecast CI level | No |
| PROPHET_SEASONALITY_MODE | additive | Prophet seasonality mode | No |
| XGBOOST_MAX_DEPTH | 6 | XGBoost tree depth | No |
| XGBOOST_LEARNING_RATE | 0.05 | XGBoost learning rate | No |
| SHAP_NUM_SAMPLES | 100 | SHAP background samples | No |
| ANOMALY_THRESHOLD | 2.5 | Z-score anomaly threshold | No |
| CACHE_TTL_HOURS | 24 | Forecast cache TTL | No |
| CLEAR_CACHE_ON_RETRAIN | true | Invalidate cache after retrain | No |
| API_KEY | — | ML Service API key (auth) | Yes |

## B. DOCKER AND KUBERNETES CONFIGURATION

**ML Service Dockerfile:**

```
FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.11 \
     /usr/local/lib/python3.11
COPY . .
RUN mkdir -p models
EXPOSE 8000
CMD ["uvicorn", "main:app",
     "--host", "0.0.0.0", "--port", "8000"]
```

**Kubernetes Deployment (excerpt):**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bi-dashboard-ml-service
  namespace: retail-services
spec:
  replicas: 2
  selector:
    matchLabels:
      app: bi-dashboard-ml-service
  template:
    spec:
      containers:
      - name: ml-service
        image: my-registry/bi-dashboard-ml-service:v1.0.0
        ports:
        - containerPort: 8000
        resources:
          requests: { cpu: 500m, memory: 1Gi }
          limits:   { cpu: 2000m, memory: 2Gi }
        volumeMounts:
        - name: models
          mountPath: /app/models
        livenessProbe:
          httpGet: { path: /health, port: 8000 }
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
```

```
          httpGet: { path: /health, port: 8000 }
          initialDelaySeconds: 10
          periodSeconds: 5
      volumes:
      - name: models
        persistentVolumeClaim:
          claimName: ml-models-pvc
```

# C. UNIT TEST CASES

**Table C.1:** Unit and integration test cases for the ML Service

| ID | Module | Test Description | Expected Outcome |
|---|---|---|---|
| UT-01 | Feature Eng. | `build_feature_matrix` with 365-day series | All 14 features present; no NaN |
| UT-02 | Feature Eng. | Promotional flag active during promo period | `promotion_active = 1` |
| UT-03 | Prophet | Train on 365-day series; predict 30 days | MAPE $\leq 20\,\%$ on synthetic data |
| UT-04 | XGBoost | Train and predict; CI bounds not inverted | `yhat_lower < yhat_upper` |
| UT-05 | Ensemble | Weight optimiser converges on known optimal $w$ | $\lvert w - w^* \rvert < 0.05$ |
| UT-06 | SHAP | Top feature = `promotion_active` when active | First contribution is promo feature |
| UT-07 | Alert | Stock 10, forecast 50 units over 7 days | Stockout risk $\geq 0.90$ (Critical) |
| UT-08 | Alert | Inject $5\sigma$ spike; $\theta = 2.5$ | Anomaly alert raised |
| UT-09 | Health API | All services up | `status = "healthy"` |
| UT-10 | Health API | DB unreachable | `status = "degraded"` |
| IT-01 | Forecast API | `POST /forecast` with valid `product_id` | HTTP 200; forecasts array non-empty |
| IT-02 | Forecast API | `POST /forecast` with unknown product | HTTP 404 |
| IT-03 | Explain API | `POST /explain` returns SHAP array | `feature_contributions` non-empty |
| IT-04 | Alert API | `POST /alerts/generate` returns scored alerts | Each alert has severity field |
| IT-05 | Train API | `POST /models/train` triggers retraining | Model file updated on disk |
| IT-06 | Circuit break | Gateway calls ML Service; ML Service returns 500 | Gateway returns 200 with cached data |

## D. SAMPLE FORECAST API REQUEST AND RESPONSE

**Request (Gateway → ML Service):**

```
POST http://localhost:8000/forecast
Content-Type: application/json
X-API-Key: <api_key>

{
  "product_id":          "item-001",
  "horizon_days":        30,
  "include_confidence": true,
  "confidence_level":    0.95
}
```

**Response (truncated to first 3 days):**

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "product_id":     "item-001",
  "model_type":     "ensemble",
  "model_version": "v1",
  "mape":           7.14,
  "generated_at":   "2025-08-01T02:05:12Z",
  "forecasts": [
    {
      "date":         "2025-08-02",
      "yhat":         142.5,
      "yhat_lower":   128.3,
      "yhat_upper":   156.7
    },
    {
      "date":         "2025-08-03",
      "yhat":         138.2,
      "yhat_lower":   124.5,
      "yhat_upper":   151.9
    },
    {
      "date":         "2025-08-04",
      "yhat":         155.8,
      "yhat_lower":   140.2,
      "yhat_upper":   171.4
    }
  ]
}
```